三、检索增强生成（RAG）



1.文档处理

在前面的学习中，我们已经掌握了与大模型交互的基本方式。无论是直接调用模型，还是通过精心设计的提示词，本质上都是向模型输入一段文本，然后获得相应的输出。

```
response = llm.invoke("请介绍一下人工智能")
```

这种方式简单直接，适合学习和演示。然而，当我们开始构建真实的 AI 应用时，会很快遇到一个新的问题：

大模型需要处理的数据，往往并不来自用户的直接输入，而是来自各种外部资源。

例如：

- 一个企业知识库问答系统，需要读取大量 PDF 合同和制度文件；

- 一个智能客服系统，需要引用产品手册和 FAQ 文档；
- 一个代码助手，需要理解整个代码仓库的内容；
- 一个法律助手，需要检索法律法规条文。

这些数据以不同的格式存在于外部世界中：

- PDF、Word、Excel
- Markdown、HTML
- 网页、数据库、API 接口
- 企业内部知识库

问题来了：我们如何把这些格式各异、来源不同的外部数据，加载到程序中，并交给大模型使用？

如果还是像之前一样手动处理，不仅麻烦，而且难以统一管理。针对这个问题，LangChain 提供了一套专门的组件 —— **Document Loader（文档加载器）**。

接下来，我们就从 **Document Loader** 开始，学习如何将外部数据接入 LangChain 体系。

1.1 外部数据的加载问题

1.1.1 真实项目中的数据来源

在实际的 AI 应用开发中，我们经常需要处理来自不同来源、不同格式的外部数据。常见的数据来源包括：

- **结构化与半结构化文档**：PDF、Word、Excel、PowerPoint
- **轻量级文本**：Markdown、TXT、JSON
- **网页内容**：HTML 页面、在线知识库
- **数据库与 API**：关系型数据库、CSV 文件、REST API 返回结果
- **企业内部系统**：Confluence、Notion、内部知识库

这些数据具有以下特点：

- 格式多样，解析方式差异大
- 内容长度不一（从几百字到几百页）
- 包含丰富的结构信息（标题、表格、页码、层级等）
- 往往带有重要的业务元数据（来源部门、创建时间、版本等）

如果我们继续使用传统的手动方式来读取这些文件，不仅开发效率低下，后续的文本处理、检索和生成也难以统一管理。因此，我们需要一套更系统、更统一的加载方案。

1.1.2 原始加载方法

在Python中，读取不同格式的数据通常使用不同的标准库或第三方库。

（1）读取普通文本文件：

```python
with open('data/中华人民共和国个人信息保护法_20210820.txt','r',encoding='utf-8') as f:
    content = f.read()
print(content)
```

（2）读取JSON文件：

```python
import json

with open('data/database.json','r',encoding='utf-8') as f:
    content = json.load(f)
print(content)
```

这两种方式对开发者来说非常自然：打开文件、解析内容、得到Python原生对象（`str` 或 `dict`）。

（3）读取 PDF 文件

当面对PDF文件时，上述方式立即失效：

```python
with open("data/制造车间质量管控技术规范.pdf", "r", encoding="utf-8") as f:
    content = f.read()
```

直接报错。PDF不是纯文本文件，它具有二进制结构、页面组织和编码信息，必须使用专门的解析库。

```
# 安装
pip install pypdf pymupdf
```

使用 `pypdf` 库的正确方式：

```python
from pypdf import PdfReader

reader = PdfReader("data/制造车间质量管控技术规范.pdf")
text = ""

for page in reader.pages:
    text += page.extract_text()
```

实际使用中可以发现，pypdf 在处理复杂技术规范类PDF时提取效果很差：表格结构丢失、文字顺序混乱、大量格式信息无法保留。

即使同为PDF文件，不同公司的解析库API也完全不同，提取质量差异显著。

（4）PyMuPDF 读取

```python
import fitz  # PyMuPDF

doc = fitz.open("data/制造车间质量管控技术规范.pdf")
text = ""

for page in doc:
    text += page.get_text()
```

两种PDF解析库不仅调用方式完全不同，提取结果的质量也存在明显差距。这意味着开发者需要为同一种文件类型维护多套解析逻辑，并根据实际效果反复切换和调试。

1.1.3 不同数据源带来的核心痛点

现实中常见的数据源及其读取方式存在明显差异。即使限定在PDF这一种文档类型内，不同解析库之间也存在完全不同的API和效果。

数据源	典型读取方式	返回类型	后续处理一致性
TXT	open()	str	低
JSON	json.load()	dict / list	中等
PDF	PdfReader()（pypdf）	自定义对象	高
PDF	fitz.open() + get_text()（PyMuPDF）	自定义对象	高
网页	requests.get() + 解析库	Response对象	极高

核心问题

不同类型的数据需要不同的库、不同的API、产生不同的返回格式。

即使数据源类型相同（例如都是PDF），不同解析库的调用方式、提取质量也存在显著差异。想要获取来源信息（文件名、页码、章节、URL等）也必须各自实现一套逻辑。当构建RAG系统时，数据切分、嵌入、检索、生成等后续步骤将因为数据源和解析方式的不同而需要重复编写大量适配代码，维护成本极高且容易出错。

1.1.4 LangChain的统一方案

LangChain的核心设计目标之一是：**让后续组件不再关心数据来源。**

无论数据来自TXT、PDF、Word、Markdown、网页还是数据库，加载方式都统一为 loader.load()，返回结果统一为 List[Document]。

示例代码：

```
from langchain_community.document_loaders import TextLoader,PyPDFLoader,PyMuPDFLoader

# 读取TXT
loader = TextLoader("data/中华人民共和国个人信息保护法_20210820.txt", encoding="utf-8")
docs = loader.load()
print(docs)
print('='*30)
# 读取PDF
loader = PyPDFLoader("data/制造车间质量管控技术规范.pdf")
docs = loader.load()
print(docs)
print('='*30)
# 读取PDF
loader = PyMuPDFLoader("data/制造车间质量管控技术规范.pdf")
```

```
docs = loader.load()
print(docs)
```

运行后会发现两点重要变化：

1. **加载方法统一**：所有数据源都通过 loader.load() 完成。
2. **返回格式统一**：无论原始数据是什么类型，最终都变成了 List[Document]。

这自然引出了下一个问题：这个Document到底是什么？它为什么能把所有数据源都统一起来？

1.2 文档类：Document

1.2.1 Document 类结构

（1）概述

`Document` 类位于 `langchain_core.documents` 模块，是 LangChain 官方定义的标准文档对象。其设计理念是：**将文本内容与元数据分离存储**，便于在整个 RAG 流程中统一传递和处理。

Document 类的核心价值：

- 实现了文本与元数据的解耦，便于后续处理。
- 所有 LangChain 组件（如 TextSplitter、VectorStore、Retriever、Loader 等）都以 Document 作为标准输入输出格式。
- 支持序列化（JSON）、批量操作、元数据过滤等高级特性，是 RAG 流水线的“通用语言”。

（2）构成

Document 是 LangChain 中用于统一表示数据的核心数据结构。它的典型结构如下：

```
Document(
    page_content="文本内容...",
    metadata={"source": "制造车间质量管控技术规范.pdf", "page": 0}
)
```

Document 主要包含两个字段：

- **page_content**：存储纯文本内容。 无论原始数据来自PDF、网页还是数据库，page_content 都只保留可用于后续处理的文本部分。这使得文本切分（Splitter）、向量化（Embedding）、检索等下游组件可以完全不关心数据来源。
- **metadata**：以字典形式保存来源信息（文件名、页码、章节、作者、日期、URL等）。 这是RAG系统中实现可溯源、可过滤、可引用生成的关键基础。没有统一的metadata，后续很难做精确的知识过滤和答案引用。

（3）重要性

LangChain并没有发明新的文件读取技术。

它真正统一了两件事：

1. **Loader接口**：将各种数据源（包括同类型但不同解析库）的加载方式抽象为统一的 loader.load() 调用。

2. **Document格式**：将各种数据的表示方式统一为 Document(page_content, metadata) 对象。

通过这两重统一，RAG pipeline的后半段（Splitter → Embedding → VectorStore → Retriever → Chain）能够用完全一致的方式处理来自任意数据源的内容，从而显著降低系统复杂度并提高可维护性。

这就是LangChain在数据接入层存在的根本价值。

1.2.2 Document 类的创建方法

在实际项目中，Document 通常有两种最常见的创建方式：

（1）直接实例化

这是最基础、最直观的创建方式，适合理解 Document 的结构。

```
from langchain_core.documents import Document

# 一个典型的 Document 示例
doc = Document(
    page_content="第一条 个人信息处理应当遵循合法、正当、必要原则……",    # 核心文本内容
    metadata={
        "source": "中华人民共和国个人信息保护法",
        "chapter": "第一章 总则",
        "article_num": "第1条",
        "chunk_id": "doc_001_001"
    }
)
print(doc.page_content[:20])   # 输出文本内容
print(doc.metadata)            # 输出元数据字典
```

（2）从文本列表批量创建

在处理多段文本时，常用列表推导式批量生成 Document 对象：

```
from langchain_core.documents import Document
texts = [
'''第一条 为了保护个人信息权益，规范个人信息处理活动，促进个人信息合理利用，根据宪法，制定本法。''',
'''第二条 自然人的个人信息受法律保护，任何组织、个人不得侵害自然人的个人信息权益。''',
'''第三条 在中华人民共和国境内处理自然人个人信息的活动，适用本法。
在中华人民共和国境外处理中华人民共和国境内自然人个人信息的活动，有下列情形之一的，也适用本法：
（一）以向境内自然人提供产品或者服务为目的；
（二）分析、评估境内自然人的行为；
（三）法律、行政法规规定的其他情形。''',
'''第四条 个人信息是以电子或者其他方式记录的与已识别或者可识别的自然人有关的各种信息，不包括匿名化处理后的信息。
个人信息的处理包括个人信息的收集、存储、使用、加工、传输、提供、公开、删除等。'''
]
docs = [Document(page_content=text,
                 metadata = {'source':'法律条文','chunk_ids':idx})
        for idx,text in enumerate(texts)]
for doc in docs:
    print(doc.page_content)
```

```
    print(doc.metadata)
    print('---')
```

说明：

在真实的 RAG 流程中，我们很少手动一条一条创建 Document。

更常见的做法是：通过 Loader 加载原始文件得到 Document，再交给 TextSplitter 进行切分，自动生成一批新的 Document 对象。这部分内容将在后续“文本切分”章节详细讲解。

1.3 常见文档加载器

在前面的内容中，我们已经理解了 Loader 的统一价值，也介绍过 TextLoader 的基本用法。本节不再重复讲解纯文本加载，而是重点介绍实际项目中更常用的 Markdown、CSV 和 HTML 加载器，并对 PDF 加载器 进行重点展开和选型分析。

1.3.1 Markdown 加载器

Markdown 是一种**轻量级标记语言**，它使用简单的符号来对文本进行格式化，例如：

- # 表示一级标题
- ## 表示二级标题
- * 或 - 表示无序列表
- **文本** 表示加粗
- `代码` 表示行内代码
- 等等

Markdown 的文件扩展名通常是 .md 或 .markdown。它本质上仍然是**纯文本文件**，但通过这些简单的标记符号，可以让文档具备清晰的结构（标题层级、列表、代码块等）。

示例：

```
from langchain_community.document_loaders import TextLoader

loader = TextLoader("data/家用电视技术说明文档.md",encoding="utf-8")
docs = loader.load()

print(len(docs))
print(docs[0].metadata)
print(docs[0].page_content[:300])
```

1.3.2 CSVLoader

CSV（Comma-Separated Values）是一种用逗号（或分号）作为分隔符的**结构化文本文件格式**，常用于存储表格数据。每一行代表一条记录，每一列代表一个字段。

在 LangChain 中，我们可以使用 CSVLoader 将 CSV 文件的每一行转换为一个 Document 对象。

```
from langchain_community.document_loaders import CSVLoader

loader = CSVLoader("data/百度模型.csv",encoding='utf-8')

docs = loader.load()

print(len(docs))
print(docs[0].page_content)
print(docs[0].metadata)
```

如果是`tsv`格式，需要把默认的分隔符变成";"

```
loader = CSVLoader("data/train.tsv",csv_args={"delimiter": ";"},encoding='utf-8')
```

1.3.3 HTML 加载器

HTML（HyperText Markup Language）是用于创建网页的标准标记语言。它通过各种标签来定义网页的结构和内容。

在线加载网页推荐使用 WebBaseLoader，WebBaseLoader 在底层会使用 **BeautifulSoup4**（简称 BS4）来解析 HTML 页面。**BeautifulSoup4 是什么？** 它是一个非常流行的 Python 库，专门用于从 HTML 和 XML 文件中**提取数据**。它能智能地解析网页结构，让我们方便地获取标题、正文、表格等内容。

安装指令：

```
pip install beautifulsoup4
```

WebBaseLoader 可以**直接通过 URL 访问并加载网页内容**，无需提前手动下载保存为本地文件，非常方便。

```
from langchain_community.document_loaders import WebBaseLoader

# 只能加载本地HTML文件
loader = WebBaseLoader("https://www.langchain-china.com/guide.html")
docs = loader.load()

print(docs[0].page_content[:500])
print(docs[0].metadata)
```

1.4 订单检测项目

1.4.1 项目背景

随着电商业务的快速发展，客服团队每天需要处理海量的订单咨询。传统的人工客服面临以下痛点：

- 用户输入随意性强，订单号和问题描述混杂在一起，难以快速提取结构化信息。
- 人工逐条识别订单号、查询数据库、回复问题，效率低下且容易出错。
- 客服系统需要同时支持多轮对话、格式校验和数据库查询，逻辑较为复杂。

本项目旨在构建一个**智能订单客服助手**，利用 LangChain 的提示词工程和输出解析器，实现从用户自然语言输入中自动提取订单号和问题描述，并结合本地数据库进行快速查询与回复。该项目是 LangChain 提示词工程 + 输出解析器 + 实际业务场景的典型综合应用。

1.4.2 项目目标

1. 实现 AI 主动发起对话，引导用户按规范格式输入。
2. 从用户随意输入中准确提取订单号（11位纯数字）和规范化的问题描述。
3. 使用 SimpleJsonOutputParser 将模型输出解析为标准 JSON 结构。
4. 根据提取结果查询本地 database.json 数据库，返回个性化客服回复。
5. 代码结构清晰、提示词稳定、具备基本的错误处理能力，为后续扩展多轮对话和 RAG 功能奠定基础。

1.4.3 技术要点

本项目重点演示以下 LangChain 核心能力：

- **Document Loader 的实际应用**：使用 TextLoader 加载 JSON 格式的数据库文件，体现“一切数据皆可通过 Loader 统一加载”的思想。
- **结构化 Prompt 设计**：使用 ChatPromptTemplate 快速构建包含 System 和 Human 角色的提示词模板。
- **可靠的结构化输出**：通过 SimpleJsonOutputParser 强制模型输出标准 JSON，避免手动解析字符串的繁琐和不稳定。
- **业务逻辑闭环**：将模型提取结果与本地数据库结合，完成从用户输入到数据查询的完整流程。
- **教学友好设计**：代码不做过度封装，每一步都清晰可见，便于初学者理解 Prompt、Parser 和业务逻辑的协作方式。

1.4.4 完整代码

```python
import json
from langchain_community.document_loaders import TextLoader
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import SimpleJsonOutputParser
from root import llm

```

```python
# ====================== 1. 使用 Loader 加载数据库 ======================
loader = TextLoader("database.json", encoding="utf-8")
docs = loader.load()

# 把加载后的文本转成 Python 字典（实际项目中可根据需要优化）
database = json.loads(docs[0].page_content)


# ====================== 2. 提取订单编号（核心业务） ======================
extract_prompt = ChatPromptTemplate.from_messages([
    ("system", """你是一个订单信息提取助手。
请从用户输入中提取订单号，并严格按以下 JSON 格式输出：
{{"order_id": "11位数字订单号"}}

如果无法提取到有效订单号，请输出：{{"error": "未识别到有效订单号"}}"""),
    ("human", "{user_input}")
])

user_input = input("\n请输入订单相关问题：")

messages = extract_prompt.format_messages(user_input=user_input)
raw_output = llm.invoke(messages)

parser = SimpleJsonOutputParser()
parsed = parser.parse(raw_output)

print("\n【模型提取结果】")
print(parsed)


# ====================== 3. 根据订单号查询数据库 ======================
if "error" in parsed:
    print("\n【查询结果】未识别到有效订单号。")
else:
    order_id = parsed["order_id"]
    order_info = database.get(order_id)

    print(f"\n【数据库查询结果】")
    if order_info:
        print(f"订单号：{order_id}")
        print(f"商品名称：{order_info.get('商品名称', '未知')}")
        print(f"下单时间：{order_info.get('下单时间', '未知')}")
        print(f"订单状态：{order_info.get('订单状态', '未知')}")
    else:
        print(f"未找到订单号 {order_id} 的信息。")
```

1.5 RAG

我们现在已经能**提取用户订单号**，并**查询到数据库里的结构化信息**（商品名称、状态等）。但如果只是把查询结果原样打印，客服回复依然是"死板的模板"**。更智能的做法是：把查到的订单信息"塞回"给大模型**，让它生成自然、贴心的回复，比如：

> "您的订单 `12345678901`（商品：机械键盘）当前状态为'运输中'，预计明天送达，感谢耐心等待～"

而上述完整的过程，就被称为 Retrieval Augmented Generation（RAG）

1.5.1 为什么要用 RAG

RAG（Retrieval-Augmented Generation，检索增强生成） 是一种让大语言模型在回答问题时，**先"翻书查资料"，再组织答案**的技术。

大语言模型（LLM）虽然很强大，但有两个天然缺陷：

（1）知识边界问题（Knowledge Boundary）

LLM 的所有知识都固化在训练参数中，存在天然的**知识边界**：

- **时效性缺失**：训练数据有明确的截止日期，无法获取实时信息（最新政策、股票价格、物流状态、新闻事件等）。
- **私有知识缺失**：企业内部文档、用户个人数据、特定领域知识库，模型在训练时从未见过。
- **长尾知识覆盖不足**：冷门、专业或小众知识在训练数据中可能极少或错误。

后果：模型要么直接拒绝回答（"我不知道"），要么基于旧知识或猜测强行回答，导致信息过时或错误。

（2）幻觉问题（Hallucination）

即使模型"认为自己知道"，它生成答案的本质机制仍然是**根据概率预测下一个词（next token prediction）**，而非基于真实数据库进行验证。

模型不会主动检查自己说的话是否真实，因此当相关知识不足或上下文缺失时，它很容易：

- 编造不存在的事实、人物、事件或数据
- 捏造参考文献、来源或链接
- 输出看似合理、逻辑自洽，但实际完全错误的答案

（3）经典例子

经典例子：客服订单查询

用户提问："我的订单现在到哪里了？"

维度	不使用 RAG（纯 LLM 生成）	使用 RAG（检索增强生成）
行为本质	模型"猜测"订单状态	先查询真实订单状态，再生成回答

维度	不使用 RAG（纯 LLM 生成）	使用 RAG（检索增强生成）
回答依据	可能产生幻觉	严格基于企业数据库/物流系统的事实
数据获取	无法访问企业实时数据	可以访问企业内部数据库或知识库
知识时效性	知识完全停留在训练阶段	可获取最新实时信息
风险	生成“您的订单正在配送途中，预计明天送达”这类听起来很合理但完全错误的答案	生成“您的订单已于今日 14:30 签收，签收人：张先生”这类有事实依据的答案

1.5.2 RAG的工作原理（三步走）

（1）检索（Retrieval）

- 根据用户输入，从**外部知识库**（可以是数据库、文档、网页、向量库等）中找到最相关的信息片段。
- 在之前的订单项目里，这一步就是根据订单号去 `database.json` 里查找对应的订单信息。

（2）增强（Augmented）

- 把检索到的信息作为“上下文”，拼接到给大模型的提示词（Prompt）中。
- 例如：`"订单信息：商品=机械键盘，状态=运输中。用户问：我的订单到哪了？请根据订单信息回答。"`

（3）生成（Generation）

- 大模型根据 **用户原始问题 + 检索到的资料**，生成最终的自然语言回复。
- 因为有了资料支撑，回答就变得准确、可溯源、且可以包含最新信息。

1.5.3 RAG的实现

```python
# ====================== 3. 查询数据库 ======================
if "error" in parsed:
    print("\n【查询结果】未识别到有效订单号。")
else:
    order_id = parsed["order_id"]
    order_info = database.get(order_id)
    print(f"\n【数据库查询结果】")

    if order_info:
        print(f"订单号：{order_id}")
        print(f"商品名称：{order_info.get('商品名称', '未知')}")
        print(f"下单时间：{order_info.get('下单时间', '未知')}")
        print(f"订单状态：{order_info.get('订单状态', '未知')}")

        # ====================== 4. RAG 生成回答 ======================
        # 把查询到的订单信息作为 context 喂给 LLM
```

```
        context = f"""订单号：{order_id}
商品名称：{order_info.get('商品名称', '未知')}
下单时间：{order_info.get('下单时间', '未知')}
订单状态：{order_info.get('订单状态', '未知')}"""

        generate_prompt = ChatPromptTemplate.from_messages([
            ("system", "你是一个专业的订单客服助手。请根据用户的问题和提供的订单信息，用中文给出准确、友好的回答。如果信息不足，请直接说明。"),
            ("human", f"""用户问题：{user_input}

订单信息：
{context}

请回答用户的问题。""")
        ])

        gen_messages = generate_prompt.format_messages()
        response = llm.invoke(gen_messages)

        print("\n【RAG 生成回答】")
        print(response.content if hasattr(response, "content") else response)

    else:
        print(f"未找到订单号 {order_id} 的信息。")
```

2. 非结构化场景下的 RAG 系统

在上一章，我们通过订单智能客服的例子，理解了 RAG 的基本工作原理：先检索外部信息，再让大模型生成回答。

然而，真实企业中大量需要回答的问题并不来自结构化数据库，而是来自公司内部的各种**非结构化文本**，比如管理制度、操作流程、售后政策、产品说明等。这些内容通常以长文本的形式存储，且大模型在训练时并未见过。

本章我们将以公司管理条例为例，探讨当知识以非结构化文本形式存在时，如何构建一个有效的 RAG 系统。

2.1 案例引入

2.1.1 案例背景与目标

背景：在实际企业运营中，人力资源部门经常需要回答员工关于请假、调休、年终奖、离职等流程的重复性问题。这些问题虽然看似简单，但由于涉及公司内部具体规定，且不同员工的提问方式各异，导致 HR 团队需要投入大量时间进行一对一解答，工作效率较低。

为了改善这一现状，某公司决定构建一个**员工自助问答系统**。该系统希望让员工能够通过自然语言提问（如“我请病假需要提前几天申请？”“我想离职要办什么手续？”），直接获得准确、规范的回答，从而减轻 HR 的重复劳动负担。

资料：公司已经将相关管理条例整理成结构化的文本形式，具体内容如下：

```
documents = [
    "请假流程：员工因病、事、年假等原因需要请假时，应提前至少1个工作日通过企业微信或OA系统提交请假申请。申请需注明请假类型、起止时间、请假事由，并上传相关证明材料（如病假需提供医院诊断证明）。部门主管在收到申请后需在24小时内审批，超过3天以上的请假需由部门经理或以上领导审批。请假期间如需延长，须提前办理续假手续。未经批准擅自离岗的，按旷工处理。",

    "调休流程：员工因加班产生的调休需在加班后30天内申请使用。申请调休时需在系统中选择对应的加班记录，并填写调休时间。调休申请需由部门主管审批，原则上应在工作安排允许的情况下进行。调休最长可累积至当年年底，逾期未使用的调休将自动清零。调休期间按正常出勤计算工资，但需确保所在团队工作不受影响。紧急情况下可临时申请调休，但需及时补交申请单。",

    "年终奖申请：符合条件的正式员工可在每年12月1日至12月15日期间通过HR系统提交年终奖申请。申请需填写个人年度工作总结、绩效自评及下一年度工作计划。年终奖发放标准根据员工职级、绩效考核结果、公司整体业绩及个人贡献综合评定。部门主管需在12月20日前完成对下属的年终奖评级推荐，人力资源部最终审核后于次年1月底前统一发放。未在规定时间内提交申请的员工视为自动放弃本年度年终奖。",

    "离职流程：员工提出离职需提前30天以书面形式（或通过OA系统）向直属主管提交离职申请。主管收到申请后应在3个工作日内与员工进行离职面谈，了解离职原因并进行挽留。员工需在离职前完成工作交接、归还公司财产（如工牌、电脑、钥匙等）、结清借款及报销。人力资源部负责办理社保、公积金、工资结算及离职证明开具等手续。未按规定办理离职手续或存在未结清事项的，公司有权暂缓办理离职手续。"
]
```

目标：基于以上非结构化文本数据，构建一个智能问答系统，使员工能够通过自然语言与系统交互，快速获取准确的管理流程信息。

2.1.2 工作流程拆解

基于公司管理条例构建的自助问答系统，其核心仍然遵循 RAG 的基本工作流程：**检索（Retrieval）→ 增强（Augmentation）→ 生成（Generation）**。但与订单查询场景不同的是，本案例中的知识以非结构化文本形式存在，且用户提问更加口语化和多样化。

下面我们对系统的工作流程进行拆解：

（1）检索（Retrieval）

当员工提出问题后，系统首先需要从公司管理条例文档库中，找到与用户问题最相关的内容片段。

例如：

- 用户提问：“我请病假需要提前几天申请？”

系统需要在 `documents` 中找到与“请假”相关的政策内容（即第一条请假流程）。

这一步是整个系统的关键难点。因为用户提问通常比较口语化，而政策原文是正式书面表达，两者之间存在语义差距。如何准确、高效地找到相关政策，是本案例需要重点解决的问题。

（2）增强（Augmentation）

检索到相关政策内容后，系统会将其作为上下文信息，与用户原始问题一起组织成完整的提示词（Prompt），发送给大语言模型。

例如，增强后的 Prompt 可能包含以下内容：

用户原始问题 + 从文档库中检索到的请假流程原文 + 回答要求（如“请严格基于提供的政策内容回答”）

通过这种方式，大模型不再仅依赖自身参数，而是能够“参考”公司内部的具体规定进行回答。

（3）生成（Generation）

大模型接收到增强后的提示词后，会综合用户问题和检索到的政策内容，生成自然、准确的回复。

例如，当用户问“我请病假需要提前几天申请？”时，系统最终可能生成如下回答：

> “根据公司请假流程，员工因病需要请假时，应提前至少1个工作日通过企业微信或OA系统提交请假申请。病假需提供医院诊断证明。请假申请由部门主管在24小时内审批。”

这个回答既自然流畅，又有明确的依据来源。

2.1.3 检索方式

在上一节的工作流程拆解中，我们看到整个 RAG 系统的成败很大程度上取决于**检索阶段**能否准确找到与用户问题相关的政策内容。在订单查询案例中，我们可以通过提取订单号进行精确匹配。但在本案例中，知识以非结构化文本形式存在，且用户提问方式多样，因此检索方式的选择变得尤为关键。

下面我们分析几种常见的检索方式在公司管理条例自助问答场景中的表现：

1. 精确匹配（结构化 ID 检索）

这种方式要求用户提供明确的标识信息（如订单号、合同编号等），系统通过唯一 ID 进行精确查找。

在当前场景下，这种方式**基本无法使用**。因为：

- 用户提问通常是自然语言（如“我想请病假”“我年底想拿奖金”），不会主动提供任何结构化 ID；
- 公司管理条例本身也没有像订单号那样的唯一结构化索引。

2. 关键词匹配

关键词匹配通过计算用户问题与政策文本中的关键词重合度来检索内容。常见实现包括 SQL LIKE、BM25 等算法。

这种方式在一定程度上可以工作，但存在明显局限：

- 当用户使用同义词或口语化表达时容易失败。例如用户问“我不想干了，能走吗？”，而政策原文使用的是“离职”，两者关键词重合度很低；
- 召回结果可能不准确或不完整，影响后续生成质量。

3. 向量语义检索

向量语义检索通过 Embedding 模型将文本转换为向量，计算语义相似度来找到相关内容，而不是依赖字面关键词。

在公司管理条例场景中，这种方式具有明显优势：

- 能够理解语义相近但表达不同的内容（如“请病假”与“因病需要请假”、“不想干了”与“离职”）；
- 对口语化提问的容错性更强，更适合真实用户的使用习惯。

检索方式	是否适用于本案例	主要问题	推荐程度
精确匹配	否	用户不会提供结构化 ID	不推荐
关键词匹配	部分适用	难以处理同义词和口语化表达	一般
向量语义检索	高度适用	需要引入 Embedding 模型和向量库	**推荐**

通过以上分析可以看出，在处理公司管理条例这类**非结构化文本知识**时，传统的精确匹配和单纯的关键词匹配都存在明显不足。向量语义检索能够更好地理解用户真实意图，是目前构建此类自助问答系统的主流选择。

接下来，我们将重点讨论如何使用向量语义检索来实现本案例中的 RAG 系统。

2.1.4 语义检索原理

向量语义检索是目前处理非结构化文本最主流的方式，其核心思想是：

> **把文本转化为高维向量，通过计算向量之间的距离来判断语义相似度，而不是依赖字面关键词的重合。**

具体原理如下：

- **Embedding（向量化）**

使用 Embedding 模型（如 OpenAI Embedding、Qianfan Embedding 等）将每条政策文本转换成一个固定长度的数值向量。这个向量能够捕捉文本的语义信息。

- **向量相似度计算**

当用户提问时，同样将其转换成向量，然后计算它与知识库中所有政策向量的相似度（通常使用余弦相似度）。相似度越高，说明语义越接近。

- **Top-K 召回**

系统返回相似度最高的 K 条政策内容，作为上下文提供给大模型。

这种方式的优势在于能够理解“请病假”和“因病需要请假”、“不想干了”和“离职”这类语义相近但字面不同的表达。

2.1.5 检索实现

（1）数据加载

```
import numpy as np
from sentence_transformers import SentenceTransformer
from langchain_core.prompts import ChatPromptTemplate
from root import llm

# ====================== 1. 准备数据 ======================
documents = [
    "请假流程：员工因病、事、年假等原因需要请假时，应提前至少1个工作日通过企业微信或OA系统提交请假申请。申请需注明请假类型、起止时间、请假事由，并上传相关证明材料（如病假需提供医院诊断证明）。部门主管在收到申请后需在24小时内审批，超过3天以上的请假需由部门经理或以上领导审批。请假期间如需延长，须提前办理续假手续。未经批准擅自离岗的，按旷工处理。",

```

```
    "调休流程：员工因加班产生的调休需在加班后30天内申请使用。申请调休时需在系统中选择对应的加班记录，并填写调休时间。调休申请需由部门主管审批，原则上应在工作安排允许的情况下进行。调休最长可累积至当年年底，逾期未使用的调休将自动清零。调休期间按正常出勤计算工资，但需确保所在团队工作不受影响。紧急情况下可临时申请调休，但需及时补交申请单。",

    "年终奖申请：符合条件的正式员工可在每年12月1日至12月15日期间通过HR系统提交年终奖申请。申请需填写个人年度工作总结、绩效自评及下一年度工作计划。年终奖发放标准根据员工职级、绩效考核结果、公司整体业绩及个人贡献综合评定。部门主管需在12月20日前完成对下属的年终奖评级推荐，人力资源部最终审核后于次年1月底前统一发放。未在规定时间内提交申请的员工视为自动放弃本年度年终奖。",

    "离职流程：员工提出离职需提前30天以书面形式（或通过OA系统）向直属主管提交离职申请。主管收到申请后应在3个工作日内与员工进行离职面谈，了解离职原因并进行挽留。员工需在离职前完成工作交接、归还公司财产（如工牌、电脑、钥匙等）、结清借款及报销。人力资源部负责办理社保、公积金、工资结算及离职证明开具等手续。未按规定办理离职手续或存在未结清事项的，公司有权暂缓办理离职手续。"
]
```

（2）文本表示（句向量化）

查询侧：输入文本 → 查询向量

文档侧：输入文本 → 文档向量

```python
# ====================== 2.向量化 ======================
print("正在加载 sentence-transformers 模型，请稍候...")
embeddings = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')  # 支持中文

# 对所有文档进行向量化（只计算一次）
print("正在对文档进行向量化...")
database = embeddings.encode(documents, convert_to_numpy=True)

query = embeddings.encode([question], convert_to_numpy=True)[0]
```

（3）语义检索（相似度计算）

计算查询向量与文档向量的相似度

核心操作：`相似度 = 余弦(查询向量, 文档向量)`

输出：相似度分数（0~1） `例：0.87`

```python
import numpy as np
def cosine_similarity(vec1, vec2):
    # 计算公式 cos = (vec1·vec2)/(||vec1||·||vec2||)
    # 计算点积
    dot_product = np.dot(vec1, vec2)

    # 计算L2范数（自动处理零向量）
    norm1 = np.linalg.norm(vec1)
    norm2 = np.linalg.norm(vec2)

    # 计算余弦相似度
    return dot_product / (norm1 * norm2)
```

```
res = []
for i in range(len(database)):
    res.append({"score":cosine_similarity(query, database[i]),
"sentence":documents[i]})
print(res)
```

(4) 取top-k个答案

```
top_k = 2
final_res = sorted(res, key=lambda x: x["score"], reverse=True)[:top_k]
for idx, item in enumerate(final_res):
    print(f'{idx + 1}. {item["sentence"]}')
```

2.1.6 生成实现

该阶段的核心是让生成模型能够有效利用检索到的外部知识，从而提升回答的准确性和信息量。

(1) 设计模板

```
system_template = """
你是一个AI助理，你的专长是人工智能的知识解答。请严格遵守以下要求：
1. 回答必须完全基于提供的文档内容，不得臆造信息
2. 直接回应问题，避免无关内容
"""

human_template = """根据以下文档内容，回答问题：
{context}

问题：{query}
答案：
"""
```

(2) 融合输入与推理：

```
chat_prompt = ChatPromptTemplate.from_messages([
    SystemMessagePromptTemplate.from_template(system_template),
    HumanMessagePromptTemplate.from_template(human_template)
])

question = input("我不想干了，我应该怎么操作")
messages = chat_prompt.format_messages(context=final_res, query=question)
response = llm.invoke(messages)
```

2.2 Chroma数据库

在前面的前面的章节中，我们使用 sentence-transformers + NumPy 实现了向量语义检索。这种方式虽然能够帮助我们理解向量检索的核心原理，但它存在一个明显的局限：所有向量数据都存储在内存中。

2.2.1 局限性

这种方式虽然简单，但在实际企业级项目中会遇到以下几个严重问题：

1. 数据持久化问题

- 程序一旦重启或崩溃，内存中的向量数据就会全部丢失。
- 每次启动程序都需要重新对所有文档进行向量化，耗时且浪费资源。
- 在实际业务中，知识库通常是长期存在的，不能依赖程序运行状态。

2. 数据规模受限

- 内存容量有限，当文档数量达到几万、几十万甚至更多时，内存很容易不足。
- 大规模向量数据的存储和检索性能会急剧下降。

3. 功能缺失

内存实现通常只支持最基础的相似度搜索，缺乏以下生产级能力：

- 元数据过滤（按部门、时间、类型等条件筛选）
- 增量更新（新增或修改文档后无需全部重新计算）
- 持久化存储和备份
- 高效的索引和检索优化

2.2.2 数据库介绍

向量数据库是专门用于存储和检索高维向量的数据库系统。目前业界常用的向量数据库主要有以下几种：

向量数据库	类型	特点	适用场景	学习难度
Chroma	本地/嵌入式	轻量级、易上手、无需额外服务	**小规模**（推荐 < 50万条）	低
FAISS	本地库	性能极高，Facebook 开源	**中小规模到大规模**	中
PGVector	PostgreSQL 扩展	直接在 PostgreSQL 中支持向量检索	**中小规模**（< 100万条）	中
Weaviate	本地/云	支持模块化扩展，功能丰富	**中等到大规模**	中
Pinecone	云服务	托管式服务，使用简单	**中等到大规模**（按需扩展）	低
Milvus	分布式	支持大规模分布式部署，功能强大	**大规模及以上**（千万级以上）	高

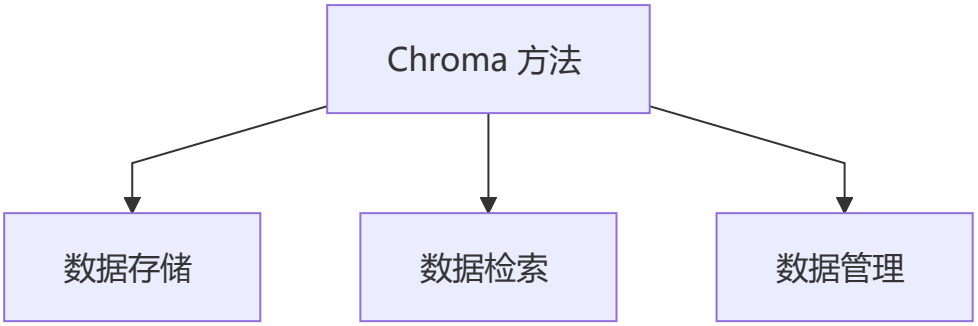
不同数据量下的选择建议：

- **小规模数据**（< 50万条）：推荐使用 **Chroma**，部署简单、学习成本低，非常适合教学、原型验证和中小型项目。
- **中等规模数据**（50万 ~ 500万条）：可以选择 **FAISS**、**Weaviate** 或 **PGVector**。
- **大规模数据**（500万条以上）：建议使用 **Milvus** 或云原生方案（如 Pinecone、Zilliz Cloud）。
- **超大规模 + 高并发**：一般选择分布式向量数据库（如 Milvus、Qdrant 集群版）。

不同的向量数据库在性能、易用性、部署方式和成本上各有侧重。对于教学和快速验证想法来说，**Chroma** 是目前最合适的选择。

2.2.3 Chroma的使用介绍

Chroma 在 LangChain 中的方法设计兼顾了易用性和灵活性，从简单的相似度搜索到复杂的元数据过滤，都能通过简洁的 API 实现。特别适合需要快速搭建原型又要求生产级性能的场景。



Chroma已经集成到langchain框架里面，也可以单独使用。

（1）部署

安装方法：

```
# terminal
pip install langchain-chroma
```

调用方法

```
from langchain_chroma import Chroma
```

（2）Client（客户端）

`Client` 是 Chroma 的**入口对象**，负责管理整个数据库的连接、持久化、Collection 创建等全局操作。

主要类型：

- `PersistentClient`：持久化客户端（推荐用于生产和开发），数据会保存在磁盘上，关闭程序后数据不会丢失。
- `HttpClient`：通过 HTTP 连接远程 Chroma 服务（用于 Server 模式）。

创建示例：

```python
import chromadb

# 最推荐的持久化客户端
client = chromadb.PersistentClient(path="./law_db")   # 数据会保存在 ./law_db 文件夹
```

（3）Collection（集合）

`Collection` 是 Chroma 中**真正存储数据的地方**，类似于传统数据库中的"表"（Table）。

每个 Collection 包含：

- 大量向量（embeddings）
- 对应的原始文本（documents）
- 元数据（metadatas）
- 向量 ID（ids）

重要特点：

- 一个 Chroma 实例可以创建多个 Collection
- 每个 Collection 可以指定不同的索引参数（如使用余弦相似度）
- Collection 支持增删改查等 CRUD 操作

（4）其他重要属性和概念

属性/概念	说明	常用参数示例
`embedding_function`	向量化的核心模型，必须在创建 Chroma 时传入	`embedding_model`（Embedding-V1、BGE 等）
`collection_name`	Collection 的名称，用于区分不同数据集	`"rag_knowledge"`
`metadata`	存储在每个 Document 上的附加信息，可用于过滤	`{"chapter": "第一章", "source": "law"}`
`ids`	每个向量的唯一标识符（必须唯一）	`"doc_001_chunk_003"`

2.2.4 创建数据库

（1）标准创建方法

```python
from langchain_chroma import Chroma
import chromadb
# 1. 创建 Client
client = chromadb.PersistentClient(path="./law_db")

# 2. 创建 Chroma 向量库（LangChain 封装版本）
db = Chroma(
    client=client,
    collection_name="law_knowledge",
    embedding_function=embeddings      # 必须传入
)
```

创建完成后，看到生成一个文件夹，具体结构如下（具体名称会有偏差）：

```
-data
|--b68ac....
|--data_level0.bin
    |--header.bin
    |--length.bin
    |--link_lists.bin
|--chroma.sqlite3
```

如果 collection_name 已存在 → 加载（Load）已有 Collection

如果 collection_name 不存在 → 自动创建（Create）一个新的 Collection

（2）快速创建方法（Document列表转换）

```python
from langchain_community.vectorstores import Chroma
from langchain_core.documents import Document

# 这里使用 4.2.2 (2)方法获取 Document 列表
texts = [
'''第一条 为了保护个人信息权益，规范个人信息处理活动，促进个人信息合理利用，根据宪法，制定本法。''',
'''第二条 自然人的个人信息受法律保护，任何组织、个人不得侵害自然人的个人信息权益。''',
'''第三条 在中华人民共和国境内处理自然人个人信息的活动，适用本法。
在中华人民共和国境外处理中华人民共和国境内自然人个人信息的活动，有下列情形之一的，也适用本法：
（一）以向境内自然人提供产品或者服务为目的；
（二）分析、评估境内自然人的行为；
（三）法律、行政法规规定的其他情形。''',
'''第四条 个人信息是以电子或者其他方式记录的与已识别或者可识别的自然人有关的各种信息，不包括匿名化处理后的信息。
个人信息的处理包括个人信息的收集、存储、使用、加工、传输、提供、公开、删除等。'''
]
docs = [Document(page_content=text,
                 metadata = {'source':'法律条文','chunk_ids':idx})
        for idx,text in enumerate(texts)]

db = Chroma.from_documents(
    documents= docs,                    # List [Document]
    embedding=embeddings,               # 你的 Embedding 模型
    persist_directory="./law_db",         # 持久化路径
```

```
    collection_name="law_knowledge",     # 集合名称（类似表名）
    ids=[f"law_chunk_{i}" for i in range(len(docs))] # 设置ids
)
```

如果 collection_name 不存在 → 创建（Create）一个新的 Collection 并写入数据

如果 collection_name 已存在 → 追加（Append） 数据到已有 Collection

基于上述逻辑，建议使用第一种标准方法，更加安全

2.2.5 数据库的增删查

（1）获取信息

```
# 获取集合信息
collection = db.get()
print(collection["documents"])
print(f'文档的数量：{len(collection["documents"])}')
```

（2）增加

```
# 方法一：添加 Document 列表（推荐）
new_docs = [
    Document(
        page_content="第十四条 个人信息处理者应当对其处理的个人信息负责...",
        metadata={"chapter": "第二章", "article": "第14条", "source": "个人信息保护法"}
    ),
    Document(
        page_content="第十五条 处理个人信息应当取得个人同意...",
        metadata={"chapter": "第二章", "article": "第15条", "source": "个人信息保护法"}
    )
]

db.add_documents(new_docs,
                 ids=['law_chunk_14','law_chunk_15'])
print("✅ 新文档已添加")
collection = db.get()
print(collection["documents"])
print(f'文档的数量：{len(collection["documents"])}')
```

（3）删除

```python
# 1. 按 id 删除
db.delete(ids=['law_chunk_1','law_chunk_2'])

# 2. 按 metadata 条件删除（推荐）
db.delete(
    where={"chapter": "第二章"}           # 删除所有第二章的文档
)
collection = db.get()
print(collection["documents"])
print(f'文档的数量：{len(collection["documents"])}')
```

2.2.6 完整实验

```python
from langchain_community.vectorstores import  Chroma
from openai import OpenAI
import pickle
import chromadb
from langchain_core.documents import Document

# 1.数据处理
# 1.1 加载数据
with open('./法律条文切片.pkl','rb') as f:
    texts = pickle.load(f)

# 1.2 创建 Document 类
docs = [Document(page_content=text,
                 metadata = {'source':'法律条文'})
        for idx,text in enumerate(texts)]

# 2.数据库操作
# 2.1 创建数据库(这里我们更换一下路径)
client = chromadb.PersistentClient(path="./final_db")

db = Chroma.from_documents(
    documents= docs,
    embedding=embeddings,
    persist_directory="./final_db",
    collection_name="law_knowledge",
    ids=[f"law_chunk_{i}" for i in range(len(docs))] # 设置ids
)

# 2.2 数据库增删查
# 2.2.1 查看数据库
collection = db.get()
print(collection["documents"][0])
print(f'文档的数量：{len(collection["documents"])}')

new_docs = [
    Document(
        page_content="第十四条 个人信息处理者应当对其处理的个人信息负责...",
```

```
        metadata={"chapter": "第二章", "article": "第14条", "source": "个人信息保护法"}
    ),
    Document(
        page_content="第十五条 处理个人信息应当取得个人同意...",
        metadata={"chapter": "第二章", "article": "第15条", "source": "个人信息保护法"}
    )
]

db.add_documents(new_docs,
                 ids=['law_chunk_101','law_chunk_102'])
collection = db.get()
print(collection['ids'])
print(f'文档的数量：{len(collection["documents"])}')

# 2.2.3 删除数据
db.delete(ids=['law_chunk_1','law_chunk_2'])
collection = db.get()
print(collection['ids'])
print(f'文档的数量：{len(collection["documents"])}')

db.delete(
    where={"chapter": "第二章"}           # 删除所有第二章的文档
)
collection = db.get()
print(collection['ids'])
print(f'文档的数量：{len(collection["documents"])}')
```

2.2.7 数据检索方法

1. `similarity_search()` - 相似度搜索（基础）

```
results = vector_db.similarity_search(
    query="信息保护",  # 查询文本
    k=3,  # 返回数量
    filter={"category": "法律"}  # 元数据过滤
)
```

2. `similarity_search_with_score()` - 带相似度得分

```
docs_scores = vector_db.similarity_search_with_relevance_scores(
    query="个人责任",
    k=2
)
for doc, score in docs_scores:
    print(f"相似度: {score:.4f}\n内容: {doc.page_content[:50]}...")
```

2.2.8 检索

（1）正常检索

```python
from langchain_community.vectorstores import Chroma
from langchain_core.runnables import RunnablePassthrough, RunnableLambda

# ==================== 1. 加载你已构建的法律数据库 ====================
vector_db = Chroma(
    persist_directory="./final_db",
    embedding_function=embeddings,      # 确保你之前创建数据库时使用的 embeddings 已定义
    collection_name="law_knowledge"
)

# ==================== 第一块：基础检索示例（已适配你的数据库） ====================

# 1. similarity_search() - 相似度搜索（基础）
results = vector_db.similarity_search(
    query="信息保护",           # 查询文本
    k=3,                          # 返回数量
    filter={"source": "法律条文"}   # 根据你数据库的 metadata 过滤
)
print("=== 基础检索结果 ===")
for i, doc in enumerate(results):
    print(f"[{i+1}] {doc.page_content[:150]}...")

# 2. similarity_search_with_relevance_scores() - 带相似度得分
docs_scores = vector_db.similarity_search_with_relevance_scores(
    query="个人责任",
    k=4
)
print("\n=== 带分数检索 ===")
for doc, score in docs_scores:
    print(f"相似度: {score:.4f} | 内容: {doc.page_content[:100]}...")
```

（2）定义函数

```python
def retrieve_docs(query,top_k=3,threshold=0.7):
    res = db.similarity_search_with_relevance_scores(query, k=top_k)
    res = [r for r in res if r[1] > threshold]
    if res:
        final_res = ''.join([r[0].page_content for r in res])
    else:
        final_res = ''
    return final_res
```

2.2.9 嵌入模板与推理

```python
from langchain.prompts import (
    ChatPromptTemplate,
    SystemMessagePromptTemplate,
    HumanMessagePromptTemplate,
    AIMessagePromptTemplate
)
system_template = """
你是一个专业的法律AI助理，专长是根据《个人信息保护法》等法律条文进行知识解答。
请严格遵守以下要求：
1. 回答必须完全基于提供的文档内容，不得臆造信息
2. 直接回应问题，避免无关内容
"""
human_template = """根据以下文档内容，回答问题：
{context}

问题：{query}
答案：
"""

chat_prompt = ChatPromptTemplate.from_messages(
    [SystemMessagePromptTemplate.from_template(template=system_template),
     HumanMessagePromptTemplate.from_template(template=human_template)]
)
query = input("请输入问题：")
context = retrieve_docs(query)

messages = chat_prompt.format_messages(context=context, query=query)
print(messages)
messages = convert(messages)
print(llm.invoke(messages))
```

2.3 作业

在课堂上，我们已经完成了基于内存 + sentence-transformers 的公司管理条例自助问答系统（2.1 节内容）。该系统能够通过语义检索找到相关政策，并让大模型生成回答。

然而，内存实现存在数据无法持久化、难以扩展等问题。在实际项目中，我们通常会使用向量数据库来解决这些问题。

本次作业要求你使用 **Chroma 向量数据库**，重新实现公司管理条例的智能问答系统。